Rate Limiter & Notification System Design

Part 1: In-Memory Rate Limiter Coding Exercise
Part 2: Scalable Notification System Architecture

Part 1 — In-Memory Rate Limiter

1. Objective

The objective is to build a reusable in-memory rate limiter that protects an internal API from abuse by allowing at most N requests during a rolling W-second window for each client key. Each API key or user ID is limited independently.

2. Sliding-Window Approach

A Map stores request timestamps for every client key. When allow(key) is called, timestamps older than the current time minus the configured window are removed. If the remaining number of requests is below N, the new timestamp is recorded and the request is allowed. Otherwise, the request is rejected.

3. Core Interface

The limiter exposes a deliberately small interface:

```
limiter.allow("user123") → true or false
```

4. Major Components

Component	Responsibility
RateLimiter	Applies the sliding-window policy and exposes allow(key).
In-memory store	Stores request timestamps per client key using a Map.
Clock	Provides the current time and can be replaced by a fake clock in tests.
Automated tests	Verify boundaries, bursts, key isolation, and window expiry.

5. Test Strategy

• First request is allowed.

• Requests up to N within the window are allowed.

• The N+1 request is rejected.

• A request becomes eligible again after the oldest request leaves the window.

• Exact window-boundary behaviour is tested explicitly.

• Different client keys do not affect one another.

• Burst traffic is handled correctly.

• Injected time makes tests deterministic without waiting in real time.

6. Concurrency and Future Distributed Storage

The exercise assumes a single process. If concurrent requests are possible, the check-and-record operation must be atomic so two requests cannot both observe available capacity and exceed the limit. In a multi-instance deployment, the in-memory Map would not be sufficient because each server would have different state. A distributed store such as Redis could provide shared state and atomic operations.

7. Tradeoffs

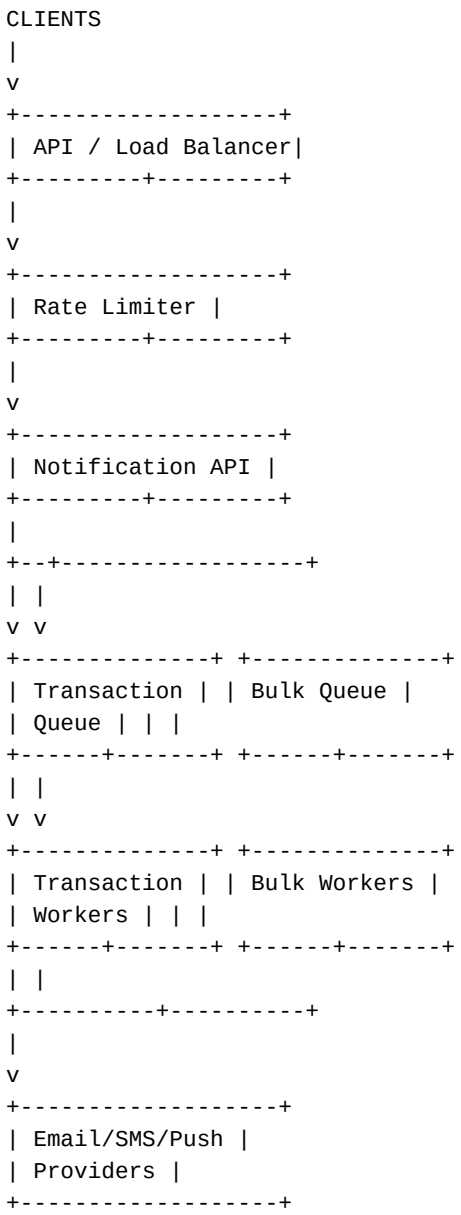
The in-memory design is simple, fast, and easy to test, but state is lost when the process restarts and cannot naturally be shared across multiple application instances. The timestamp-list approach is easy to understand and appropriate for the exercise, while a production implementation may need more memory-efficient structures, cleanup of idle keys, synchronization, or distributed storage.

Part 2 — Notification System Design

1. System Objective

The notification platform must reliably process transactional and bulk notifications while protecting the API from abuse. It should support retries, failover, de-duplication, prioritisation, and horizontal scaling without allowing large bulk workloads to delay important transactional messages.

2. Architecture



3. Major Components

Component	Purpose
Load Balancer / API	Accepts requests and distributes traffic across application instances.
Rate Limiter	Prevents excessive requests from individual clients.
Notification API	Validates requests, creates notification jobs, and assigns priority.
Transactional Queue	Handles urgent notifications such as OTPs, alerts, and account events.
Bulk Queue	Handles campaigns, newsletters, and other high-volume workloads.

Workers	Consume queue messages and communicate with external notification providers.
Database	Stores notification metadata, status, idempotency keys, and audit information.
External Providers	Deliver email, SMS, or push notifications to recipients.

4. Data Stores

A relational database can store notification records, delivery status, client information, and idempotency records. Queues provide durable asynchronous processing. In a scaled deployment, Redis or another distributed store can be used for shared rate-limiter state, short-lived locks, or caching.

5. Retry Strategy

Transient provider failures should be retried using exponential backoff with jitter. Each job should have a maximum retry count. After repeated failures, the job should move to a dead-letter queue for investigation or controlled reprocessing. Permanent failures, such as invalid recipient addresses, should not be retried indefinitely.

6. Failover

Application servers and workers should be stateless and deployed as multiple instances. If an instance fails, another instance can continue processing. Durable queues prevent messages from being lost during temporary worker failures. Multiple notification providers can also be configured so delivery can fail over to another provider when appropriate.

7. De-duplication

Each notification request should contain a unique idempotency key or event ID. The system stores this identifier before or atomically with job creation. If the same request arrives again, the existing job is returned or ignored instead of creating another notification. Consumers should also be designed to handle duplicate delivery attempts safely.

8. Transactional vs Bulk Prioritisation

Transactional notifications receive higher priority because they are usually time-sensitive. Separate queues prevent bulk campaigns from consuming all worker capacity. Dedicated transactional workers or reserved capacity can ensure that OTPs, security alerts, and important account notifications continue to flow even during a large campaign.

9. Key Tradeoffs

• Separate queues improve reliability and prioritisation but increase operational complexity.

• Asynchronous processing improves API responsiveness but means delivery is not always immediate.

• Retries improve reliability but can create duplicates unless idempotency is implemented.

• In-memory rate limiting is fast and simple but does not work correctly across multiple application instances.

• A distributed store improves scalability but introduces network dependency and additional infrastructure.

10. Future Improvements

With more time, the system could add distributed rate limiting, automatic key cleanup, provider health monitoring, circuit breakers, delivery metrics, tracing, autoscaling workers, stronger concurrency controls, and an operational dashboard. The rate limiter could also support interchangeable policies such as token bucket in addition to sliding window.

Conclusion

The proposed design keeps the coding exercise small and testable while providing a clear path toward a production-ready notification architecture. The main principles are isolation of responsibilities, deterministic testing, asynchronous processing, idempotency, controlled retries, failure recovery, and protection of high-priority transactional traffic.